

From an idea in your head to a machine that runs it

Lecture notes on what programming actually is, how a computer system carries out your instructions, and the chain of translators that turn readable code into something a CPU can execute — ending in Java.

MIDTERM COVERAGE 9 SECTIONS 7 DIAGRAMS

CONTENTS

- 01 Computer programming
- 02 How computer programs work
- 03 Programming languages
- 04 Translators
- 05 Types of translators
- 06 The program development process
- 07 History of Java
- 08 JDK, JRE, JVM
- 09 Errors and debugging

Computer programming

Programming is just writing instructions that a computer follows.

The computer is dumb. It does not think, it does not guess. It only does exactly what you tell it, in the exact order you tell it. Programming is you telling it what to do.

Design → Write → Test → Maintain

That's the whole job. You don't write it once and walk away — you keep fixing and updating it.

Why do we need a language for this? Because you think in ideas, and the computer only understands on/off electricity. The programming language sits in the middle and translates your idea into something the machine can run.

You → Programming Language → Computer

WHERE YOU ALREADY SEE THIS

- **ATM.** You insert a card, code checks the PIN, asks the bank's database if you have money, counts out the cash, prints the receipt. Every step is someone's code.
- **Browsers and apps.** Chrome or WhatsApp is code sending requests to a server, drawing text and images on your screen, and reacting to your taps.
- **Traffic lights.** Not magic, just a timer loop. Or sensors under the road detecting a car, then the code decides when to switch.
- **Online shopping.** Code adds up your cart, applies tax and discount, talks to the card company, then subtracts the item from warehouse stock.

THE POINT

Programming isn't just websites. Anything that reacts to input is running code.

PROGRAMMERS

Programs don't appear on their own. Humans make them — we call them **computer programmers**.

A programmer is someone who uses logic plus a specialized language to boss the hardware around. They write it, revise it, test it, update it. That's the loop.

How computer programs work

COMPUTER SYSTEM

A computer is a programmable electronic machine.

INPUT (data) → PROCESS → OUTPUT (in the format you wanted)

Two words worth remembering there:

- **Digital electronic machine** — it runs on electricity, 0s and 1s.
- **Needs to be programmed** — by itself it does nothing. It's a body with no brain until you give it instructions.

A computer system = **hardware** + **software**. You need both.

HARDWARE

Hardware is the physical stuff. If you can touch it, it's hardware — keyboard, RAM, CPU, screen, cables.

EASIEST WAY TO REMEMBER

If the computer is a human body, hardware is the bones, muscles, and organs.

Hardware gives the computer the ability to do things. But it doesn't know *what* to do. A body with no instructions just sits there.

*"Tangible components that work together
with software to make the computer function."*

SOFTWARE

Software is the instructions. You can't touch it — you can only see it working on screen.

Going back to the body analogy: if hardware is the body, software is the thoughts telling the body to move.

Why software matters:

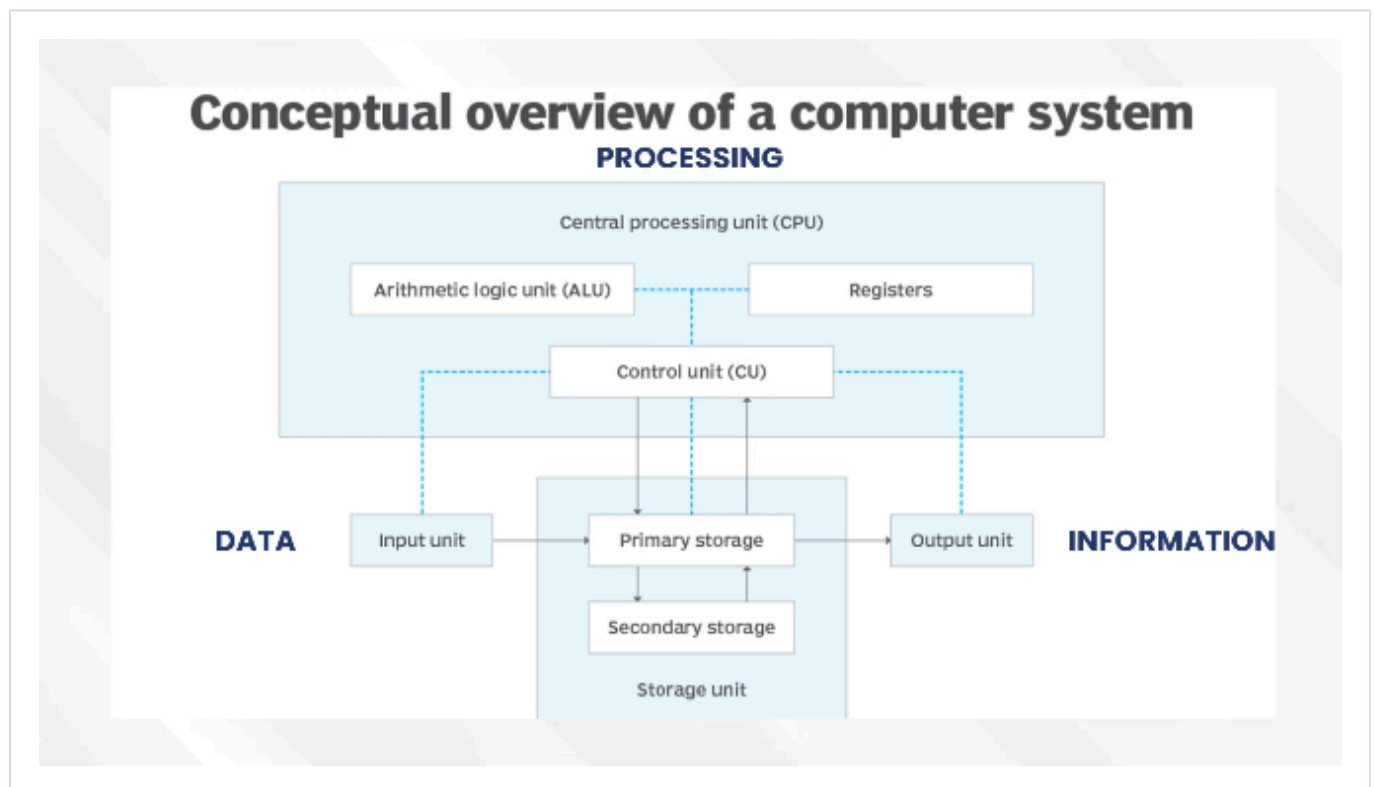
- It tells the hardware how to actually do a task.
- It makes the computer useful — work, games, chatting.
- You can update or replace it *without* buying new hardware. Same laptop, new Windows.

"The programs or instructions that tell hardware what to do."

THE FOUR UNITS OF A COMPUTER

By function, a computer is grouped into four:

- 1 **Input device** — where data comes in (keyboard, mouse, mic)
- 2 **CPU** — the brain, does the actual processing
- 3 **Memory / storage** — where data is kept
- 4 **Output device** — where the result comes out (screen, speaker, printer)



Conceptual overview of a computer system. Data enters left, information leaves right.

Reading that diagram

DATA enters the input unit → goes to the CPU → comes out as INFORMATION.

DATA VS INFORMATION

Data is the raw stuff you typed in. Information is the processed result that actually means something.

Inside the CPU:

- **ALU** (Arithmetic Logic Unit) — does the math and the comparisons.
- **Registers** — tiny, extremely fast storage sitting right beside the ALU.
- **CU** (Control Unit) — the manager, tells everything else when to move.

Storage unit:

- **Primary storage** (RAM) — fast, temporary, dies when you shut down.
- **Secondary storage** (HDD/SSD) — slower, permanent, survives shutdown.

Everything the CPU works on has to pass through primary storage first. That's why the arrows go Input → Primary storage → Output.

Programming languages

A programming language is the middle man between you and the machine.

Instead of flipping millions of on/off switches yourself, you write statements that look like English — `read`, `write`, `add` — and the language handles the switches for you.

That's why programming got easier over time. Old programmers actually had to think in switches. Now you just type `print("hi")`.

TWO TYPES

LOW-LEVEL

Close to the machine (Assembly). Fast, but painful to read.

HIGH-LEVEL

Close to humans (Java, Python, C++). Easy to read, slightly slower.

RULE OF THUMB

The closer to the machine, the faster it runs but the harder it is on you. The closer to humans, the easier on you but you give up some control and speed.

MACHINE LANGUAGE

The actual language the CPU speaks. Pure binary, only 0 and 1. Also called **machine code** or **object code**.

- Humans basically can't read it in raw form.
- The CPU executes it directly, no translation needed. This is the finish line — every language you write eventually becomes this.

ASSEMBLY LANGUAGE

One step above binary. Still low-level, still talks directly to hardware.

Instead of 0s and 1s it uses **mnemonics** — short word-ish symbols like `Add`, `Sub`, `Mul`. So it's readable by humans, barely.

It's the middle step between a high-level language like C++ and pure binary — it uses hexadecimal and binary values.

But the computer still doesn't understand it, so you need an **assembler** to convert assembly → machine code.

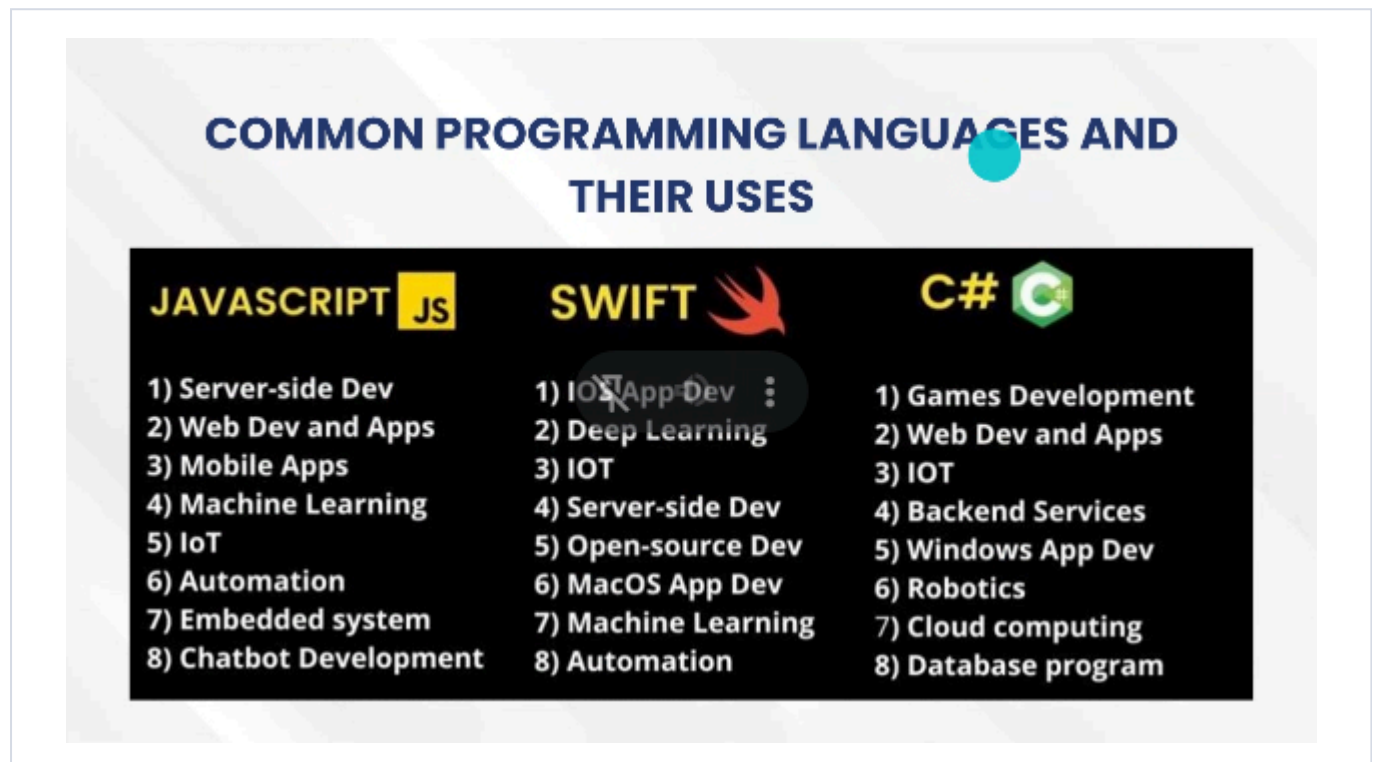
HIGH-LEVEL LANGUAGE

The generation we actually use. Written so *both* humans and computers can work with it.

Easy for humans because it uses real words, symbols, and phrases to express logic instead of raw machine operations.

Example: Java, Python, C++, JavaScript, Ruby

COMMON LANGUAGES AND WHAT THEY'RE USED FOR



JavaScript, Swift, and C# – eight common uses each.

JavaScript Web dev and apps, server-side, mobile apps, machine learning, IoT, automation, embedded systems, chatbots. Basically the language of the web that leaked everywhere else.	Swift iOS apps, macOS apps, deep learning, IoT, server-side, open-source, machine learning, automation. Apple's language.	C# Game development, web apps, IoT, backend services, Windows apps, robotics, cloud computing, database programs. Microsoft's language, big in Unity.
--	---	---

COMMON PROGRAMMING LANGUAGES AND THEIR USES

PYTHON 	JAVA 	C++ 
1) Data Science 2) Machine Learning 3) Web Development 4) Automation 5) Game Development 6) Data analysis 7) Data visualization 8) Artificial intelligence	1) Android Apps 2) Server-Side Apps 3) Enterprise Apps 4) Web Based Apps 5) Big data 6) Game Development 7) Internet of things 8) Cloud computing	1) Games Development 2) GUI Apps 3) OS 4) Database Systems 5) Embedded 6) Networking 7) Virtual Reality 8) Computer Vision

Python, Java, and C++ – eight common uses each.

Python

Data science, machine learning, web dev, automation, game dev, data analysis, data visualization, AI. The easy one — king of data and AI.

Java

Android apps, server-side, enterprise apps, web apps, big data, game dev, IoT, cloud computing. The corporate workhorse, runs anywhere.

C++

Game dev, GUI apps, operating systems, database systems, embedded, networking, VR, computer vision. The fast one, used when performance matters.

NOTICE

They overlap a lot. Most languages *can* do most things — you pick based on what it's best at and what already has support for it.

Translators

The computer only understands 0s and 1s. You don't write 0s and 1s. So something has to convert your code — that's a **translator**.

A translator is a program that turns source code into machine code. Yes — a program that helps you make programs.

"The translated machine-level code may be produced as object code."

SOURCE CODE VS OBJECT CODE

SOURCE CODE

What you wrote. Human readable. The `.java` or `.py` file.

OBJECT CODE

What came out of the translator. Low-level, machine readable, executed by the CPU directly.

Source code → translator → object code

You can read source code easily because it's meant for humans. You can't read object code because it's stored in machine/binary format.

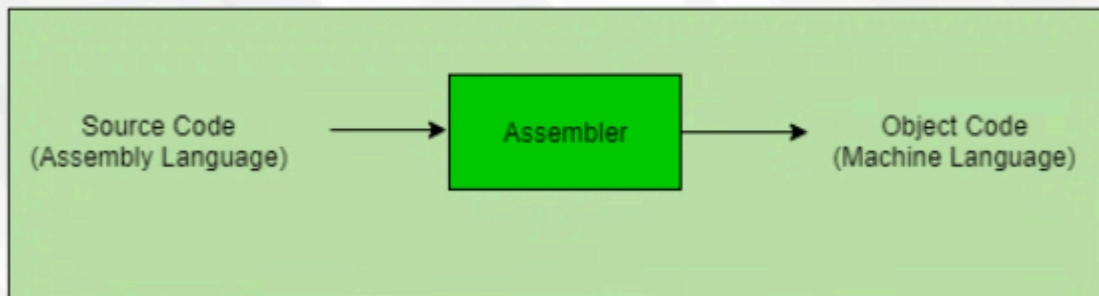
Types of translators

ASSEMBLER

A system software that converts **assembly language** instructions into machine code the CPU can execute directly.

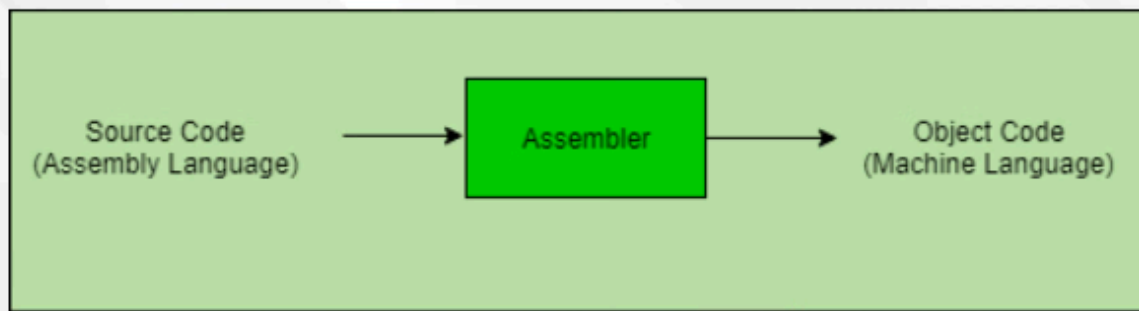
ASSEMBLER

- An assembler is a system software or translator that converts assembly language instructions into machine code (binary code) that can be directly executed by the CPU.



Source code (assembly) → assembler → object code (machine language).

ASSEMBLER



A programmer writes instructions using mnemonics and symbols in assembly language to represent machine operations.

The assembler translates assembly language instructions into machine code (binary digits 0s and 1s) that the CPU can understand and execute.

After translation, the program becomes object code, which can be executed by the computer.

The same flow, with what happens at each stage.

- 1 The programmer writes instructions using mnemonics and symbols in assembly to represent machine operations.
- 2 The assembler translates those into machine code — binary digits 0s and 1s — that the CPU understands and executes.
- 3 What comes out is object code, and the computer can now run it.

REMEMBER

An assembler is *only* for assembly language. It's not what translates Python or Java.

COMPILER

Converts the **entire** high-level source code into machine code all at once, *before* the program runs. So: translate everything, then run.

- If your code breaks the language rules, it lists *all* the errors at once.
- Much faster to run than an interpreter, since translating is already done beforehand.
- Downside — getting 50 errors dumped on you at once is hard to debug.

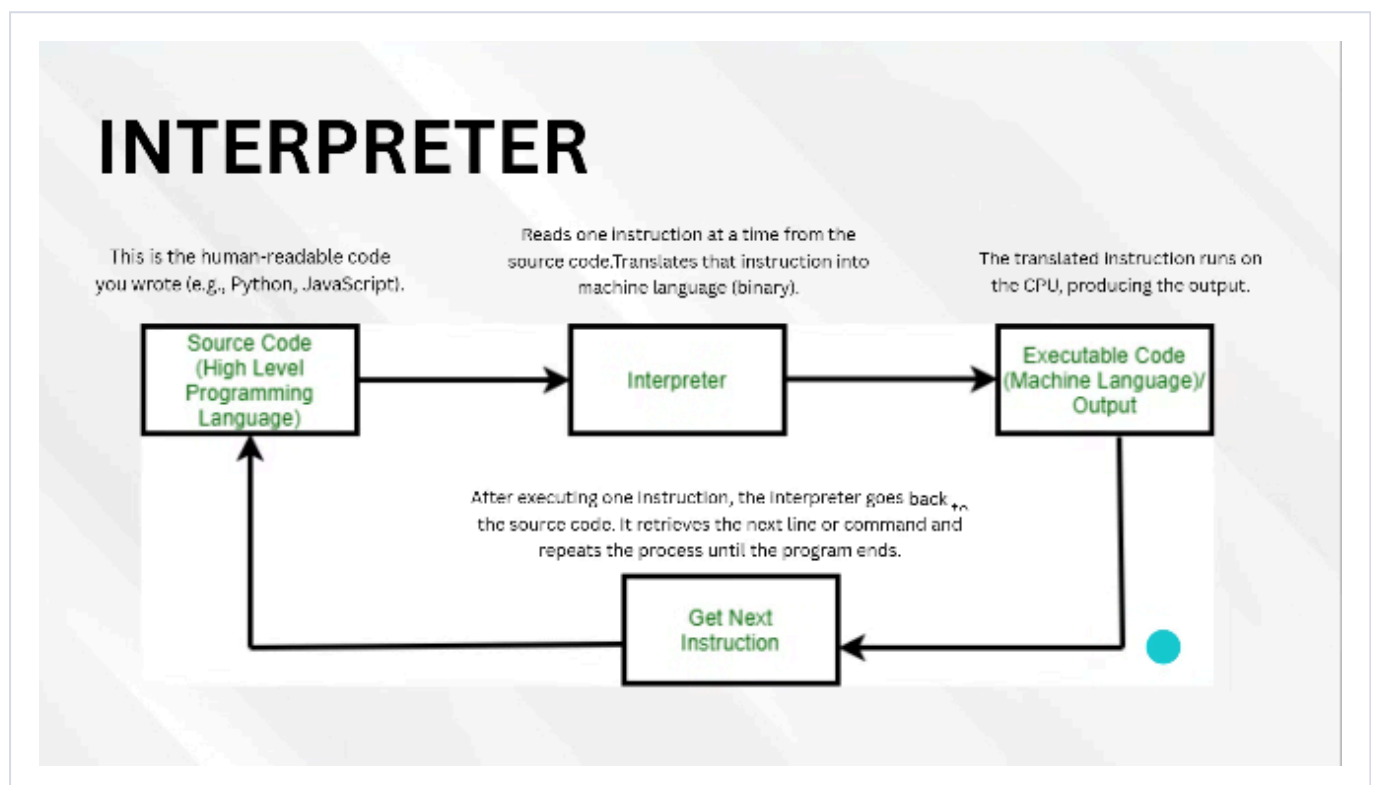
Compiled languages: C, C++, Objective C, Swift, Pascal

INTERPRETER

Same job as a compiler, but it goes **line by line** instead of all at once.

- It runs each line as it translates it.
- The moment it hits an error, it stops right there and reports it.
- Easy to debug, because you know exactly which line died.
- But slower overall, because it's translating while running.

Interpreted languages: Python, Ruby, JavaScript, Perl



The interpreter loop - it comes back for the next instruction every time.

- 1 Source code is what you wrote (Python, JavaScript).
- 2 The interpreter reads *one* instruction and translates it to binary.
- 3 That instruction runs on the CPU and produces output.
- 4 The interpreter goes back and grabs the next line. Repeat until the program ends.

COMPILER

Translates everything, then runs. Faster to run, but errors come in one big pile.

INTERPRETER

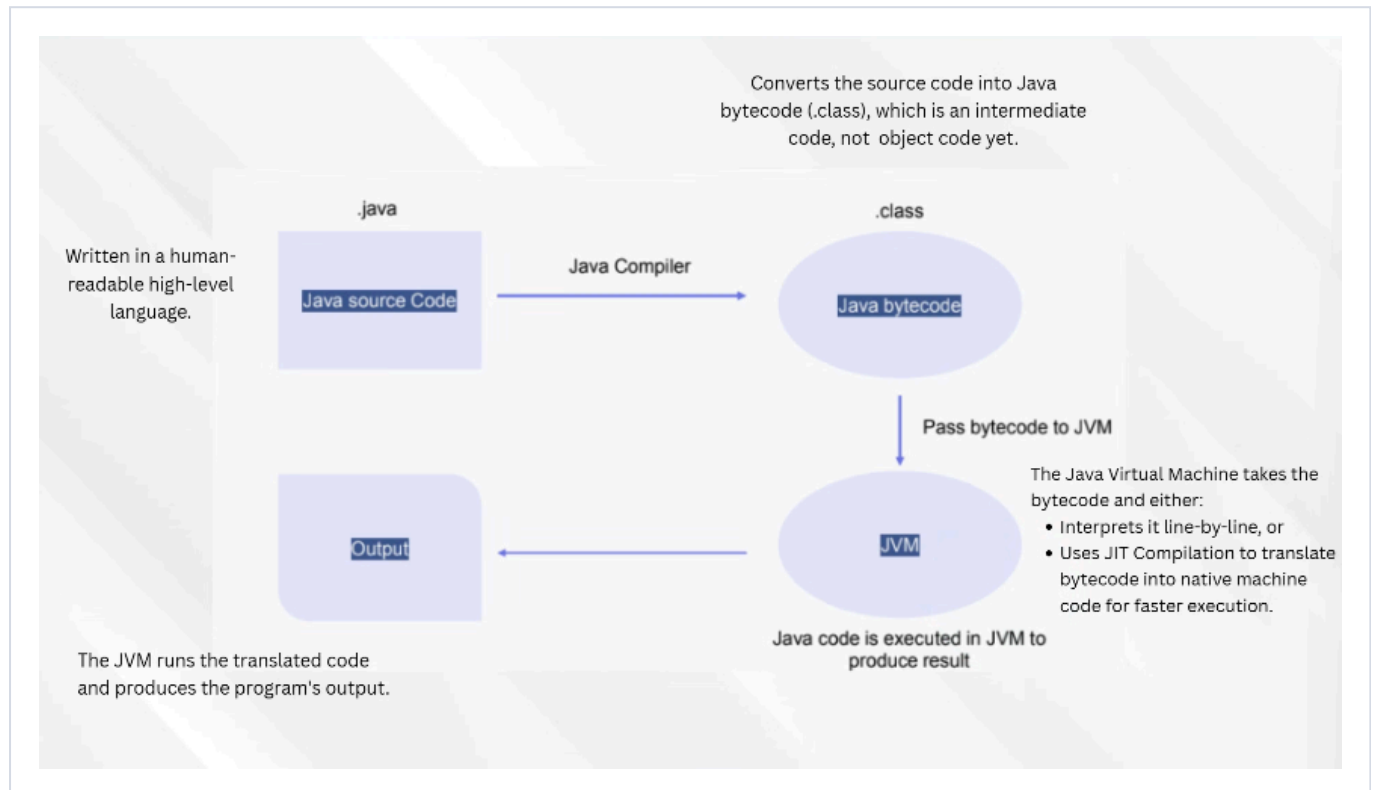
Translates and runs one line at a time. Slower, but errors are easy to find.

HYBRID

A combination of both. This is what Java does.

It compiles your source code into **bytecode** — not machine code yet — then a runtime engine interprets that bytecode. For Java that engine is the **JVM** (Java Virtual Machine).

- Like a **compiler** — it translates the whole program into an intermediate form first.
- Like an **interpreter** — it executes that intermediate code line-by-line or block-by-block at runtime.



Java's hybrid path: `.java` → compiler → `.class` bytecode → JVM → output.

- 1 You write `.java` — human readable, high level.
- 2 The Java compiler turns it into `.class` bytecode. Note: bytecode is *not* object code yet, it's an in-between form.
- 3 The bytecode gets passed to the JVM.
- 4 The JVM either interprets it line by line, or uses **JIT** (Just In Time) compilation to turn it into native machine code for speed.
- 5 The JVM runs it and you get your output.

More on bytecode

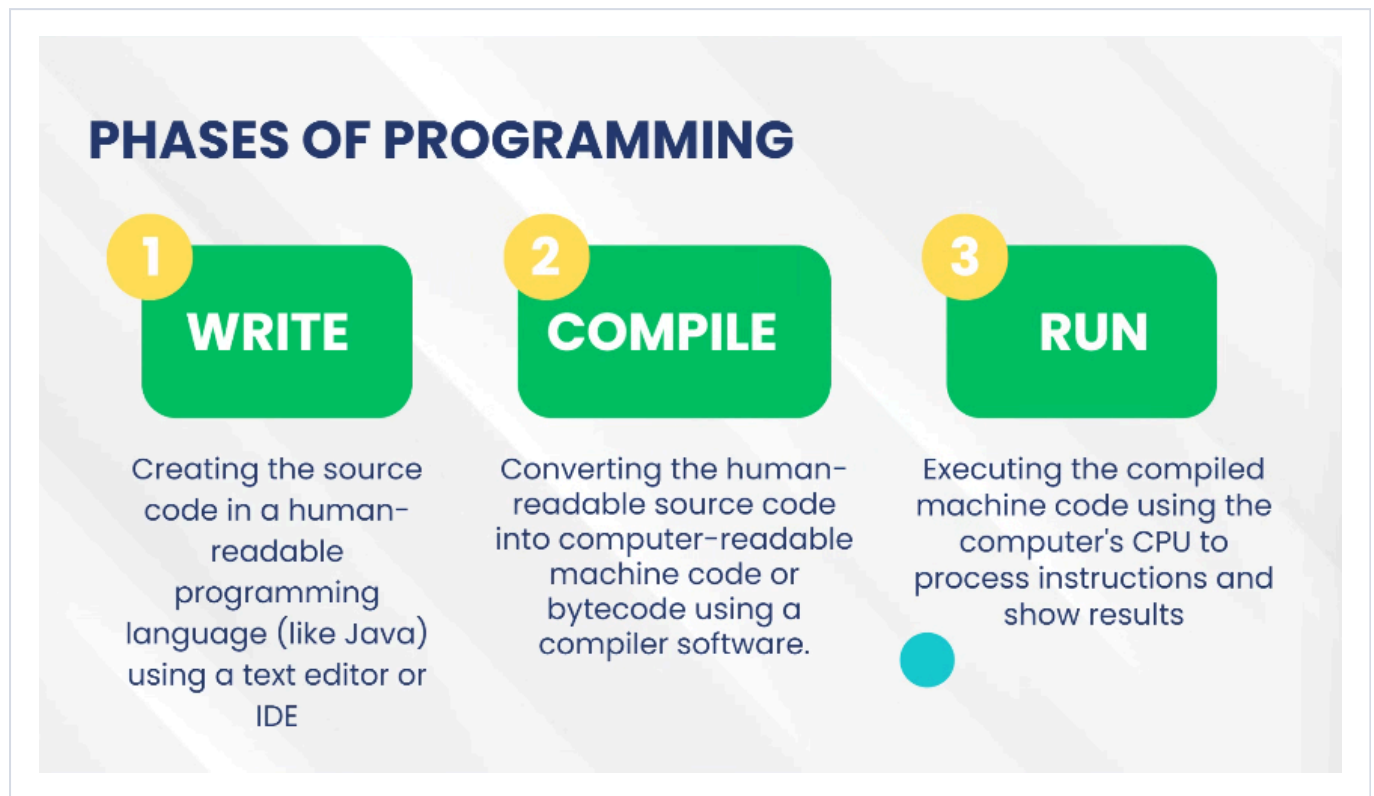
- Bytecode is not the machine language of any specific computer, but it's close to the machine language of most computers.

- The JVM is the interpreter that translates and executes that bytecode.
- Because of this, Java bytecode runs on *any* computer that has a JVM. That's the whole "write once, run anywhere" thing.
- That's also why you usually type two commands — one to compile into bytecode, one to run it. The run command tells the JVM to execute the bytecode.

WHY BOTHER WITH THIS EXTRA STEP

Because you compile once and the same `.class` file works on Windows, Mac, and Linux. With a normal compiler you'd have to recompile for each one.

The program development process



The three phases of programming.

- 1 **Write** — you type the source code in a human-readable language (like Java) using a text editor or IDE.
- 2 **Compile** — a compiler converts your source into machine code or bytecode.
- 3 **Run** — the CPU executes it and you finally see the result.

If you skip compile, there's nothing for the CPU to run. If compile fails, you go back to write. You will do this loop hundreds of times.

History of Java

- Made by **James Gosling** in the early 1990s.
- The goal wasn't even computers — they wanted a language for digital devices like set-top boxes and TVs. They considered using C++ but rejected it.
- Project name: the **Green Project**. Early language name: **Greentalk**, file extension **.gt**
- Renamed to **Oak**, after an oak tree Gosling could see from his office.
- Had to drop Oak too — Oak Technologies already trademarked it.
- Final name: **JAVA**, named after a type of espresso bean, because he thought of it while drinking coffee near his office. That's it. That's the reason.
- Built on these principles: robust, portable, platform independent, high performance, multithread.
- *TIME* magazine called it one of the Ten Best Products of 1995.

JAVA ITSELF

- General, all-purpose language. Class-based and object-oriented.
- Designed to have as few application dependencies as possible — meaning it doesn't need much extra installed to run.
- Java 1.0 released 1995. Oracle calls the open version the **JDK**.
- As of August 2026 the latest is **Java 26** (Java SE 26).
- Sun Microsystems made it. Oracle bought Sun in 2010 and now steers Java.

JDK, JRE, JVM

These three confuse everyone. Simplest version:

```
JDK contains JRE contains JVM

JDK = for making Java programs
JRE = for running Java programs
JVM = the thing that actually executes them
```

JDK – JAVA DEVELOPMENT KIT

The full package for **developing** Java apps. Contains the Java compiler, development tools, libraries, and documentation.

You need this if you want to write, compile, and build Java programs. It includes the JRE too — obviously, since you need to run your app to test it.

JRE – JAVA RUNTIME ENVIRONMENT

The package for **running** Java apps. Includes the JVM plus the libraries and components needed for execution.

This is for people who just want to run a Java app, not build one. No compiler here.

JVM – JAVA VIRTUAL MACHINE

The part that actually **executes** the bytecode.

- It's platform independent — write your Java once, run it on any system that has a JVM.
- It manages memory and handles **garbage collection** (automatically cleaning up memory you aren't using anymore).
- It provides the runtime services that keep your program running efficiently.

SUMMARY

- The **JRE** gives the environment to execute Java apps.
- The **JVM** executes the bytecode and manages runtime operations.
- The **JDK** is for developing, and it includes the compiler.

- The **compiler** turns your readable Java into bytecode.
- The **JVM** acts as the interpreter that executes that bytecode.

Errors and debugging

Whenever the compiler or interpreter hits an invalid statement — a syntax error, or you using the language wrong — it throws an error message at you.

Then you fix it and translate again. Compile or interpret, see what breaks, fix, repeat.

Finding and fixing all your syntax errors is the **first part** of debugging. Debugging means freeing the program of all errors.

NOTE THE WORDS "FIRST PART"

Syntax errors are the easy ones — the compiler literally points at them. The harder ones are **logic errors**, where the code runs fine but gives the wrong answer. Nothing points those out for you.

Write → Compile → Error → Fix → Compile → Run

That loop is programming. You will spend more time debugging than writing.